Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

# Лабораторная работа №3 по курсу

# «Операционные системы»

Группа: М8О-212БВ-25

Студент: Минин А.В.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 08.11.26

Москва, 2026

# Постановка задачи

**Вариант 11.**

Родительский процесс создает два дочерних процесса. Перенаправление стандартных потоков ввода-вывода показано на картинке выше. Child1 и Child2 можно «соединить» между собой дополнительным каналом. Родительский и дочерний процесс должны быть представлены разными программами. Родительский процесс принимает от пользователя строки произвольной длины и пересылает их в pipe1. Процесс child1 и child2 производят работу над строками. Child2 пересылает результат своей работы родительскому процессу. Родительский процесс полученный результат выводит в стандартный поток вывода. Child1 переводит строки в верхний регистр. Child2 превращает все пробельные символы в символ «_». Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в одной из двух операционных систем. В результате работы программа (основной процесс) должен создать для решение задачи один или несколько дочерних процессов. Взаимодействие между процессами осуществляется через системные сигналы/события и/или через отображаемые файлы (memory-mapped files).

# Общий метод и алгоритм решения

Использованные системные вызовы:

- shm_open(const char *name, int oflag, mode_t mode) - создаёт или открывает именованный объект разделяемой памяти и возвращает файловый дескриптор. Родитель вызывает его с O_RDWR | O_CREAT | O_TRUNC, потомки с O_RDWR;
- ftruncate(int fd, off_t length) - задаёт размер объекта. Родитель устанавливает 4096 байт (SHM_SIZE);
- mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset) - отображает объект в адресное пространство процесса. Используются PROT_READ | PROT_WRITE и MAP_SHARED, поэтому изменения видны всем процессам;
- munmap(void *addr, size_t length) - снимает отображение памяти;
- shm_unlink(const char *name) - удаляет имя объекта. Сам объект освобождается, когда закрыты все дескрипторы и отображения;
- close(int fd) - закрывает файловый дескриптор;
- sem_open(const char *name, int oflag, ...) - создаёт или открывает именованный семафор. При создании принимает дополнительно mode_t mode и unsigned int value. Родитель создаёт семафоры с начальным значением 0, потомки только открывают их;
- sem_wait(sem_t *sem) - если значение семафора больше 0, уменьшает его на 1. Иначе блокирует процесс до тех пор, пока значение не станет положительным;
- sem_post(sem_t *sem) - увеличивает значение семафора на 1 и пробуждает один ожидающий процесс;
- sem_close(sem_t *sem) - закрывает дескриптор семафора в текущем процессе;
- sem_unlink(const char *name) - удаляет имя семафора из системы.

Программа реализует конвейерную обработку строк: пользователь вводит строку, она проходит через два дочерних процесса, каждый из которых выполняет свою трансформацию, и возвращается родителю для вывода.

Между процессами создано три канала. Каждый канал состоит из объекта разделяемой памяти размером 4096 байт и семафора. Формат буфера: первые 4 байта содержат длину данных

(uint32_t), далее идёт текст. Значение UINT32_MAX является служебным маркером завершения работы.

Для передачи информации о готовности данных используются семафоры. Три семафора создаются со значением 0, то есть изначально закрыты. Семафор канала открывается (sem_post) только тогда, когда производитель записал данные в буфер. Потребитель выполняет sem_wait и блокируется, пока данных нет.

Инициализация родительского процесса происходит в несколько этапов:

- Получает свой PID через getpid и формирует из него имена объектов функцией make_name.
- Создаёт три объекта разделяемой памяти (shm_open, ftruncate), отображает их (mmap) и обнуляет поле длины в каждом буфере.
- Создаёт три семафора со значением 0 (sem_open).
- Дважды вызывает fork. В каждом потомке execv запускает child1 или child2 с PID родителя в качестве аргумента.
- Выводит PID обоих потомков и приглашение к вводу.

Основной цикл родительского процесса:

- read считывает строку со стандартного ввода.
- Если прочитано 0 байт или первый символ равен \n (пустая строка), в буфер p-c1 записывается UINT32_MAX, делается sem_post и цикл завершается.
- Иначе в буфер записываются длина и текст, затем делается sem_post на семафоре p-c1.
- Родитель блокируется на sem_wait семафора c2-p до готовности результата.
- Родитель печатает Result:  и текст из буфера c2-p, затем обнуляет длину и переходит к следующей итерации.

Цикл работы первого подпроцесса (child1):

- Открывает разделяемую память и семафоры по PID родителя и отображает память.
- Блокируется на sem_wait семафора p-c1.
- Читает длину n из входного буфера. Если n == UINT32_MAX, записывает этот маркер в выходной буфер c1-c2, делает sem_post и завершается.
- Иначе переводит каждый символ в верхний регистр (toupper), обнуляет длину во входном буфере, копирует результат в буфер c1-c2 и делает sem_post на семафоре c1-c2.

Цикл работы первого подпроцесса (child2):

- Аналогично открывает ресурсы и блокируется на sem_wait семафора c1-c2.
- Получив UINT32_MAX, завершает работу.
- Иначе заменяет каждый пробельный символ (isspace) на _, копирует результат в буфер c2-p и делает sem_post на семафоре c2-p, пробуждая родителя.

# Код программы

**parent.c**

```
#include <stdint.h>
#include <stdbool.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>

#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <semaphore.h>
#include <sys/wait.h>

#define SHM_SIZE 4096

static void make_name(char *out, const char *prefix, int pid)
{
	size_t p = 0;
	while (prefix[p]) { out[p] = prefix[p]; ++p; }
	char tmp[12]; int i = 0;
	if (pid == 0) tmp[i++] = '0';
	while (pid > 0) { tmp[i++] = (char)('0' + pid % 10); pid /= 10; }
	while (i > 0) out[p++] = tmp[--i];
	out[p] = '\0';
}

static ssize_t print_uint(int fd, uint64_t n)
{
	char buf[32];
	size_t i = 0;
	if (n == 0) { char z = '0'; return write(fd, &z, 1); }
	while (n > 0) { buf[i++] = (char)('0' + (n % 10)); n /= 10; }
	for (size_t j = 0; j < i / 2; ++j) {
		char t = buf[j]; buf[j] = buf[i - 1 - j]; buf[i - 1 - j] = t;
	}
	return write(fd, buf, i);
}

static int create_shm(const char *name)
{
	int fd = shm_open(name, O_RDWR | O_CREAT | O_TRUNC, 0600);
	if (fd == -1) {
		const char msg[] = "error: failed to create SHM\n";
		write(STDERR_FILENO, msg, sizeof(msg) - 1);
		exit(EXIT_FAILURE);
	}
	if (ftruncate(fd, SHM_SIZE) == -1) {
		const char msg[] = "error: failed to resize SHM\n";
		write(STDERR_FILENO, msg, sizeof(msg) - 1);
		exit(EXIT_FAILURE);
	}
	return fd;
}

static char *map_shm(int fd)
{
	char *p = mmap(NULL, SHM_SIZE, PROT_READ | PROT_WRITE,
			MAP_SHARED, fd, 0);
	if (p == MAP_FAILED) {
		const char msg[] = "error: failed to map SHM\n";
		write(STDERR_FILENO, msg, sizeof(msg) - 1);
		exit(EXIT_FAILURE);
```

```
	}
	return p;
}

static sem_t *create_sem(const char *name)
{
	sem_t *s = sem_open(name, O_RDWR | O_CREAT | O_TRUNC, 0600, 0);
	if (s == SEM_FAILED) {
		const char msg[] = "error: failed to create semaphore\n";
		write(STDERR_FILENO, msg, sizeof(msg) - 1);
		exit(EXIT_FAILURE);
	}
	return s;
}

static const char CHILD1_PROGRAM[] = "child1";
static const char CHILD2_PROGRAM[] = "child2";

int main(void)
{
	int ppid = (int)getpid();
	char pid_str[16];
	{
		int i = 0; int p = ppid;
		char tmp[12]; int j = 0;
		if (p == 0) tmp[j++] = '0';
		while (p > 0) { tmp[j++] = (char)('0' + p % 10); p /= 10; }
		while (j > 0) pid_str[i++] = tmp[--j];
		pid_str[i] = '\0';
	}

	char p2c1_shm_name[64], c1c2_shm_name[64], c2p_shm_name[64];
	char p2c1_sem_name[64], c1c2_sem_name[64], c2p_sem_name[64];

	 make_name(p2c1_shm_name, "/labs-p2c1-",  ppid);
	make_name(c1c2_shm_name, "/labs-c1c2-",  ppid);
	make_name(c2p_shm_name,  "/labs-c2p-",   ppid);
	make_name(p2c1_sem_name, "/labs-sem-p-c1-", ppid);
	make_name(c1c2_sem_name, "/labs-sem-c1-c2-", ppid);
	make_name(c2p_sem_name,  "/labs-sem-c2-p-",  ppid);

	 int p2c1_fd = create_shm(p2c1_shm_name);
	 int c1c2_fd = create_shm(c1c2_shm_name);
	 int c2p_fd  = create_shm(c2p_shm_name);

	 char *p2c1_buf = map_shm(p2c1_fd);
	 char *c1c2_buf = map_shm(c1c2_fd);
	 char *c2p_buf  = map_shm(c2p_fd);

	 *(uint32_t *)p2c1_buf = 0;
	 *(uint32_t *)c1c2_buf = 0;
	 *(uint32_t *)c2p_buf  = 0;

	 sem_t *p2c1_sem = create_sem(p2c1_sem_name);
	 sem_t *c1c2_sem = create_sem(c1c2_sem_name);
	 sem_t *c2p_sem  = create_sem(c2p_sem_name);

	 pid_t child1 = fork();
	 if (child1 == 0) {
		char *args[] = { (char *)CHILD1_PROGRAM, pid_str, NULL };
		execv(CHILD1_PROGRAM, args);
		const char msg[] = "error: failed to exec child1\n";
		write(STDERR_FILENO, msg, sizeof(msg) - 1);
		exit(EXIT_FAILURE);
	 } else if (child1 == -1) {
```

```
	const char msg[] = "error: failed to fork child1\n";
	write(STDERR_FILENO, msg, sizeof(msg) - 1);
	exit(EXIT_FAILURE);
}

pid_t child2 = fork();
if (child2 == 0) {
	char *args[] = { (char *)CHILD2_PROGRAM, pid_str, NULL };
	execv(CHILD2_PROGRAM, args);
	const char msg[] = "error: failed to exec child2\n";
	write(STDERR_FILENO, msg, sizeof(msg) - 1);
	exit(EXIT_FAILURE);
} else if (child2 == -1) {
	const char msg[] = "error: failed to fork child2\n";
	write(STDERR_FILENO, msg, sizeof(msg) - 1);
	exit(EXIT_FAILURE);
}

{
	const char m1[] = "Parent started.\nChild1 PID: ";
	write(STDOUT_FILENO, m1, sizeof(m1) - 1);
	print_uint(STDOUT_FILENO, (uint64_t)child1);
	const char m2[] = "\nChild2 PID: ";
	write(STDOUT_FILENO, m2, sizeof(m2) - 1);
	print_uint(STDOUT_FILENO, (uint64_t)child2);
	const char m3[] = "\nType text to get started (empty line to exit):\n";
	write(STDOUT_FILENO, m3, sizeof(m3) - 1);
}

bool running = true;
while (running) {
	char buf[SHM_SIZE - sizeof(uint32_t)];
	ssize_t bytes = read(STDIN_FILENO, buf, sizeof(buf));
	if (bytes == -1) {
		const char msg[] = "error: failed to read stdin\n";
		write(STDERR_FILENO, msg, sizeof(msg) - 1);
		exit(EXIT_FAILURE);
	}

	bool eof = (bytes == 0 || buf[0] == '\n');

	{
		uint32_t *len = (uint32_t *)p2c1_buf;
		char     *txt = p2c1_buf + sizeof(uint32_t);
		if (eof) {
			*len = UINT32_MAX;
		} else {
			*len = (uint32_t)bytes;
			memcpy(txt, buf, (size_t)bytes);
		}
	}
	sem_post(p2c1_sem);

	if (eof) { running = false; break; }
	sem_wait(c2p_sem);
	{
		uint32_t *len = (uint32_t *)c2p_buf;
		char     *txt = c2p_buf + sizeof(uint32_t);
		if (*len != 0 && *len != UINT32_MAX) {
			const char pre[] = "Result: ";
			write(STDOUT_FILENO, pre, sizeof(pre) - 1);
			write(STDOUT_FILENO, txt, *len);
			write(STDOUT_FILENO, "\n", 1);
			*len = 0;
		}
```

```
            }
     }

     pid_t exitd1 = wait(NULL);
    {
        print_uint(STDOUT_FILENO, exitd1);
        const char msg[] = " exited.\n";
        write(STDOUT_FILENO, msg, sizeof(msg) - 1);
    }
    pid_t exitd2 = wait(NULL);
    {
        print_uint(STDOUT_FILENO, exitd2);
        const char msg[] = " exited.\n";
        write(STDOUT_FILENO, msg, sizeof(msg) - 1);
    }

     sem_unlink(p2c1_sem_name);
     sem_unlink(c1c2_sem_name);
     sem_unlink(c2p_sem_name);
     sem_close(p2c1_sem);
     sem_close(c1c2_sem);
     sem_close(c2p_sem);

     munmap(p2c1_buf, SHM_SIZE);
     munmap(c1c2_buf, SHM_SIZE);
     munmap(c2p_buf,  SHM_SIZE);
     shm_unlink(p2c1_shm_name);
     shm_unlink(c1c2_shm_name);
     shm_unlink(c2p_shm_name);
     close(p2c1_fd);
     close(c1c2_fd);
     close(c2p_fd);

     write(STDOUT_FILENO, "Parent exited.\n", 15);
     return 0;
}
```

**child1.c**

```
#include <stdint.h>
#include <stdbool.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <semaphore.h>

#define SHM_SIZE 4096

static void make_name(char *out, const char *prefix, int pid)
{
        size_t p = 0;
        while (prefix[p]) { out[p] = prefix[p]; ++p; }
        char tmp[12]; int i = 0;
        if (pid == 0) tmp[i++] = '0';
        while (pid > 0) { tmp[i++] = (char)('0' + pid % 10); pid /= 10; }
        while (i > 0) out[p++] = tmp[--i];
        out[p] = '\0';
}

int main(int argc, char *argv[])
{
        if (argc != 2) {
```

```
            const char msg[] = "error: child1 expects parent pid argument\n";
            write(STDERR_FILENO, msg, sizeof(msg) - 1);
            exit(EXIT_FAILURE);
    }
    int ppid = atoi(argv[1]);

    char in_shm_name[64],  out_shm_name[64];
    char in_sem_name[64],  out_sem_name[64];
    make_name(in_shm_name,  "/labs-p2c1-",  ppid);
make_name(out_shm_name, "/labs-c1c2-",  ppid);
make_name(in_sem_name,  "/labs-sem-p-c1-", ppid);
make_name(out_sem_name, "/labs-sem-c1-c2-", ppid);

    int in_shm = shm_open(in_shm_name, O_RDWR, 0);
    if (in_shm == -1) {
            const char msg[] = "error: failed to open input SHM\n";
            write(STDERR_FILENO, msg, sizeof(msg) - 1);
            exit(EXIT_FAILURE);
    }
    int out_shm = shm_open(out_shm_name, O_RDWR, 0);
    if (out_shm == -1) {
            const char msg[] = "error: failed to open output SHM\n";
            write(STDERR_FILENO, msg, sizeof(msg) - 1);
            exit(EXIT_FAILURE);
    }

    char *in_buf = mmap(NULL, SHM_SIZE, PROT_READ | PROT_WRITE,
                MAP_SHARED, in_shm, 0);
    if (in_buf == MAP_FAILED) {
            const char msg[] = "error: failed to map input SHM\n";
            write(STDERR_FILENO, msg, sizeof(msg) - 1);
            exit(EXIT_FAILURE);
    }
    char *out_buf = mmap(NULL, SHM_SIZE, PROT_READ | PROT_WRITE,
                 MAP_SHARED, out_shm, 0);
    if (out_buf == MAP_FAILED) {
            const char msg[] = "error: failed to map output SHM\n";
            write(STDERR_FILENO, msg, sizeof(msg) - 1);
            exit(EXIT_FAILURE);
    }

    sem_t *in_sem = sem_open(in_sem_name, O_RDWR);
    if (in_sem == SEM_FAILED) {
            const char msg[] = "error: failed to open input semaphore\n";
            write(STDERR_FILENO, msg, sizeof(msg) - 1);
            exit(EXIT_FAILURE);
    }
    sem_t *out_sem = sem_open(out_sem_name, O_RDWR);
    if (out_sem == SEM_FAILED) {
            const char msg[] = "error: failed to open output semaphore\n";
            write(STDERR_FILENO, msg, sizeof(msg) - 1);
            exit(EXIT_FAILURE);
    }

    bool running = true;
    while (running) {
            sem_wait(in_sem);

            uint32_t *in_len  = (uint32_t *)in_buf;
            char     *in_text = in_buf + sizeof(uint32_t);

            uint32_t *out_len  = (uint32_t *)out_buf;
            char     *out_text = out_buf + sizeof(uint32_t);

            uint32_t n = *in_len;
            if (n == UINT32_MAX) {
```

```
                running = false;
                *in_len = 0;
                *out_len = UINT32_MAX;
                sem_post(out_sem);
        } else if (n > 0) {
                for (uint32_t i = 0; i < n; ++i)
                        in_text[i] = (char)toupper((unsigned char)in_text[i]);
                *in_len = 0;

                *out_len = n;
                memcpy(out_text, in_text, n);
                sem_post(out_sem);
        } else {
                *in_len = 0;
        }
    }

    sem_close(in_sem);
    sem_close(out_sem);
    munmap(in_buf, SHM_SIZE);
    munmap(out_buf, SHM_SIZE);
    close(in_shm);
    close(out_shm);
    return 0;
}
```

**child2.c**

```
#include <stdint.h>
#include <stdbool.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <semaphore.h>

#define SHM_SIZE 4096

static void make_name(char *out, const char *prefix, int pid)
{
    size_t p = 0;
    while (prefix[p]) { out[p] = prefix[p]; ++p; }
    char tmp[12]; int i = 0;
    if (pid == 0) tmp[i++] = '0';
    while (pid > 0) { tmp[i++] = (char)('0' + pid % 10); pid /= 10; }
    while (i > 0) out[p++] = tmp[--i];
    out[p] = '\0';
}

int main(int argc, char *argv[])
{
    if (argc != 2) {
        const char msg[] = "error: child2 expects parent pid argument\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }
    int ppid = atoi(argv[1]);

    char in_shm_name[64],  out_shm_name[64];
    char in_sem_name[64],  out_sem_name[64];
    make_name(in_shm_name,  "/labs-c1c2-",  ppid);
  make_name(out_shm_name, "/labs-c2p-",   ppid);
  make_name(in_sem_name,  "/labs-sem-c1-c2-", ppid);
  make_name(out_sem_name, "/labs-sem-c2-p-",  ppid);
```

```
int in_shm = shm_open(in_shm_name, O_RDWR, 0);
if (in_shm == -1) {
	const char msg[] = "error: failed to open input SHM\n";
	write(STDERR_FILENO, msg, sizeof(msg) - 1);
	exit(EXIT_FAILURE);
}
int out_shm = shm_open(out_shm_name, O_RDWR, 0);
if (out_shm == -1) {
	const char msg[] = "error: failed to open output SHM\n";
	write(STDERR_FILENO, msg, sizeof(msg) - 1);
	exit(EXIT_FAILURE);
}

char *in_buf = mmap(NULL, SHM_SIZE, PROT_READ | PROT_WRITE,
		MAP_SHARED, in_shm, 0);
if (in_buf == MAP_FAILED) {
	const char msg[] = "error: failed to map input SHM\n";
	write(STDERR_FILENO, msg, sizeof(msg) - 1);
	exit(EXIT_FAILURE);
}
char *out_buf = mmap(NULL, SHM_SIZE, PROT_READ | PROT_WRITE,
		MAP_SHARED, out_shm, 0);
if (out_buf == MAP_FAILED) {
	const char msg[] = "error: failed to map output SHM\n";
	write(STDERR_FILENO, msg, sizeof(msg) - 1);
	exit(EXIT_FAILURE);
}

sem_t *in_sem = sem_open(in_sem_name, O_RDWR);
if (in_sem == SEM_FAILED) {
	const char msg[] = "error: failed to open input semaphore\n";
	write(STDERR_FILENO, msg, sizeof(msg) - 1);
	exit(EXIT_FAILURE);
}
sem_t *out_sem = sem_open(out_sem_name, O_RDWR);
if (out_sem == SEM_FAILED) {
	const char msg[] = "error: failed to open output semaphore\n";
	write(STDERR_FILENO, msg, sizeof(msg) - 1);
	exit(EXIT_FAILURE);
}

bool running = true;
while (running) {
	sem_wait(in_sem);

	uint32_t *in_len  = (uint32_t *)in_buf;
	char     *in_text = in_buf + sizeof(uint32_t);

	uint32_t n = *in_len;
	if (n == UINT32_MAX) {
		running = false;
		*in_len = 0;
	} else if (n > 0) {
		for (uint32_t i = 0; i < n; ++i)
			if (isspace((unsigned char)in_text[i]))
				in_text[i] = '_';
		*in_len = 0;

		uint32_t *out_len  = (uint32_t *)out_buf;
		char     *out_text = out_buf + sizeof(uint32_t);
		*out_len = n;
		memcpy(out_text, in_text, n);
		sem_post(out_sem);
	} else {
		*in_len = 0;
```

```
            }
        }

        sem_close(in_sem);
        sem_close(out_sem);
        munmap(in_buf, SHM_SIZE);
        munmap(out_buf, SHM_SIZE);
        close(in_shm);
        close(out_shm);
        return 0;
}
```

# Протокол работы программы

Тестирование:

```
Parent started.
Child1 PID: 41937
Child2 PID: 41938
Type text to get started (empty line to exit):
1
Result: 1_
foo
Result: FOO_
12 334  4
Result: 12_334_4_

41937 exited.
41938 exited.
Parent exited.
```

# Вывод

В ходе работы был реализован конвейер из трёх процессов, обменивающихся данными через разделяемую память POSIX и синхронизируемых именованными семафорами. Работа позволила освоить создание процессов через fork и execv, использование shm_open, ftruncate и mmap для разделяемой памяти и семафоров POSIX для синхронизации.